Learning Memory System

Condensed Study Notes

Generated: 2026-05-24 23:53 UTC

COMPUTER ARCHITECTURE AND ORGANIZATION

Computer architecture and organization are two related but distinct fields that describe how computers work.

- **Computer Architecture** focuses on the **functional behavior** of a computer system as seen by a programmer. It deals with the attributes of a system that have a direct impact on the logical execution of a program. Think of it as the **blueprint** for how the computer should behave.
- **Computer Organization** deals with the **operational units** and their interconnections that realize the architectural specifications. It's about how those blueprints are actually **built and put together**.

Essentially, architecture defines *what* the computer does, while organization defines *how* it does it.

Analogy: Imagine building a house.

- **Architecture** is like the architect's drawings: it specifies the number of rooms, their sizes, where the doors and windows go, and how the rooms connect (e.g., a kitchen next to the dining room). This is what the person living in the house experiences.
- **Organization** is like the construction crew's work: it involves deciding on the specific materials (wood, brick, concrete), how to wire the electricity, how to plumb the water system, and how to lay the foundation. This is how the house is actually built to meet the architect's specifications.

COMPUTER MEMORY SYSTEM

The computer memory system is a crucial component that stores data and instructions for the CPU (Central Processing Unit) to access. Think of it like a vast library for the computer, where every book (data or instruction) has a unique address.

Memory can be broadly categorized into two main types based on speed and cost:

- **Primary Memory (Main Memory):** This is the fastest and most expensive type of memory. It's directly accessible by the CPU and holds the data and instructions that the computer is currently working with. RAM (Random Access Memory) is a common example of primary memory. It's like the desk in front of you where you keep the books you're actively reading.
- **Secondary Memory:** This type of memory is slower and less expensive than primary memory. It's used for long-term storage of data and programs that are not currently in use. Examples include hard drives and SSDs (Solid State Drives). This is like the bookshelves in your library, holding many more books but taking longer to retrieve.

The computer memory system is organized hierarchically, meaning there are different levels of memory, each with its own speed, capacity, and cost. This hierarchy is designed to balance the need for fast access with the need for large storage capacity at a reasonable price.

Learning Objectives

At the end of this module, you will be able to:

- Understand the main characteristics of computer memory systems and the use of memory hierarchy. (This builds on our previous discussion about primary and secondary memory, introducing the idea of organizing memory in different levels.)
- Understand cache memory, and the key elements of cache design. (Cache memory is a special, very fast type of memory that helps speed up your computer.)

Outline

This section outlines the key areas we will explore regarding computer memory systems.

We will delve into:

- **Cache Memory Principles**: Understanding how cache memory works to speed up access to frequently used data.
- **Elements of Cache Design**: Examining the different components and decisions involved in creating an effective cache.
- **Organization, DRAM and SRAM**: Discussing the physical organization of memory and the two main types of semiconductor memory: Dynamic Random-Access Memory (DRAM) and Static Random-Access Memory (SRAM).
- **The Memory Hierarchy**: Further exploring the layered structure of memory, from fast, small caches to slower, larger main memory and secondary storage.
- **The Memory System**: A broader look at how all the different memory components work together.
- **Interleaved Memory**: Investigating a technique used to improve memory access speed by dividing memory into parts.
- **Memory Characteristics**: Analyzing the key properties that define different types of memory.
- **Semiconductor Main Memory**: Focusing on the integrated circuits that form the primary memory of a computer.
- **Types of ROM (Read-Only Memory)**: Looking at different forms of non-volatile memory.
- **Chip Logic**: Understanding the internal workings and decision-making capabilities within memory chips.
- **Chip Packaging**: Examining how memory chips are housed and connected.

3.1 Memory Characteristics

Memory can be described by several key characteristics that help us understand how it works and how to measure its performance. These characteristics include its location, capacity, unit of transfer, method of access, performance metrics, and physical type.

1. Location:

• **Internal Memory:** Resides within the computer's main system, including registers and cache memory. The processor also has its own local memory.

• **External Memory:** Peripheral storage devices like disks and tapes, accessed via I/O controllers.

2. Capacity:

• This refers to the total amount of data a memory unit can store.

• Internal memory capacity is often expressed in **Bytes** (where 1 Byte = 8 bits) or **words**. A **word** is the "natural" unit of memory organization, often matching the number of bits used for integers or instructions (though exceptions exist).

• External memory capacity is typically measured in Bytes or larger storage units.

3. Unit of Transfer:

• For **internal memory**, it's the number of bits transferred into or out of the memory module at once. This can be equal to the word length or larger (e.g., 64, 128, 256 bits).

• For **external memory**, data is usually transferred in much larger chunks called **blocks**.

4. Method of Accessing:

This describes the sequence or order in which data can be retrieved.

• **Sequential Access:** Data is organized into **records**, and access requires moving through them in a specific linear order. Imagine reading a book page by page from the beginning to find a specific sentence.

• **Direct Access:** Similar to sequential access with a shared read/write mechanism, but records have unique physical addresses. Access involves moving to the general vicinity and then sequentially searching or counting to find the exact location. Think of a music cassette tape where you have to fast-forward or rewind to get to a specific song.

• **Random Access:** Every addressable location has a unique, built-in addressing mechanism. Access time is constant and independent of the order of previous accesses. Any location can be selected directly. This is like a digital music player where you can instantly jump to any song you want.

• **Associative Access:** A type of random access where memory is searched based on a portion of its content, not its address. All words are compared simultaneously to find a match. This is like searching your computer files by typing in a keyword, and the system finds all files containing that keyword, regardless of their location.

5. Performance:

Key metrics for evaluating memory performance:

• **Access Time (Latency):** The time taken to complete a read or write operation. For random access memory, it's the time from presenting an address to data being available. For sequential access, it's the time to position the read/write mechanism.

• **Memory Cycle Time:** Primarily for random-access memory, it's the access time plus any additional time needed before a second access can start. This accounts for things like signal stabilization or data regeneration.

• **Transfer Rate (R):** The speed at which data can be moved into or out of memory, measured in bits per second (bps). For random-access memory, it's often 1/(cycle time).

6. Physical Types:

Common types include semiconductor, magnetic surface (disks, tapes), and optical/magneto-optical memories.

7. Physical Characteristics:

- **Volatile Memory:** Information is lost when power is turned off (e.g., most semiconductor RAM).
- **Nonvolatile Memory:** Information is retained even without power (e.g., magnetic surface memories).

Computer Memory

Computer memory is a crucial system component responsible for storing information that a computer or digital device needs for immediate use. Think of it like a workspace for your computer – the more organized and accessible your tools (data and instructions) are, the faster you can get things done.

While often used interchangeably with 'primary storage' or 'main memory' in everyday conversation, computer memory is a fundamental part of the CPU (Central Processing Unit). The variety of needs across different digital devices means no single memory technology can meet all demands. Therefore, computer systems employ a 'hierarchy of memory subsystems' – a layered approach to storing information.

These memory subsystems can be categorized in several ways, including:

- **Location:** Whether they are internal (directly accessible by the CPU, like processor registers, cache, and main memory) or external (accessed via I/O modules, like optical disks, magnetic disks, and tapes).
- **Performance:** Metrics such as access time (how long it takes to retrieve data) and transfer rate (how quickly data can be moved).
- **Physical Type:** Whether they are semiconductor-based (like DRAM and SRAM), magnetic (like hard drives), optical (like CDs/DVDs), or magneto-optical.
- **Organization:** How data is structured, such as by words or bytes, and the unit of transfer (e.g., a block).
- **Physical Characteristics:** Whether the memory is volatile (loses data when power is off) or nonvolatile, and if it's erasable or non-erasable.
- **Access Method:** How data is retrieved, including sequential (reading data in order), direct (accessing specific locations), random (accessing any location with equal time), or associative (searching based on content).

The characteristics listed above, along with others like capacity and cycle time, help describe the capabilities of different memory devices.

3.2 The Memory Hierarchy

Computer systems face a challenge when it comes to memory: we want a lot of storage (high capacity) that is also very fast to access, but these two qualities are usually at odds. Generally, memories with a high capacity are slower and cheaper per bit, while memories that are fast to access are smaller and more expensive per bit.

To overcome this, computer designers use a **memory hierarchy**. This is an organized structure of different memory types, arranged in levels, to minimize the time it takes for the processor to access data and instructions.

As you move down the memory hierarchy (from the processor towards the longest-term storage), the following trends occur:

- **Capacity increases**: Each level can hold more data than the one above it.
- **Access time increases**: It takes longer to retrieve information from lower levels.
- **Cost per bit decreases**: Storing data becomes cheaper at lower levels.
- **Frequency of access decreases**: The processor needs to access data from lower levels less often.

This organization is based on a principle called **locality of reference**. This principle states that during program execution, the processor tends to access a small, clustered set of memory locations repeatedly over short periods. Think of it like repeatedly looking up the same few pages in a cookbook while preparing a specific recipe.

At the very top of this hierarchy are the **registers** inside the processor. These are the fastest, smallest, and most expensive memories, holding data the CPU is actively working with.

Below registers is **cache memory**, which is a smaller, faster memory that acts as a buffer between the processor and the main memory. It stores frequently accessed data from main memory to speed up subsequent requests. The cache is typically not directly visible to the programmer or even the processor itself; it's an automatic performance enhancement.

Main memory (often called RAM) is the primary storage for programs and data. It's larger and slower than cache but still volatile (meaning its contents are lost when power is turned off).

Finally, at the bottom are **external mass storage devices** like hard disks and removable media. These are the largest, slowest, and cheapest memories, used for long-term storage of programs and data files. They are also known as secondary or auxiliary memory and are usually accessed in terms of files or records, not individual bytes or words. Disk storage can also be used to create **virtual memory**, which extends main memory.

3.3 Cache Memory

Cache memory acts as a high-speed buffer between the CPU and main memory. Its primary goal is to combine the fast access times of expensive, small memory with the large capacity of slower, less expensive memory.

Think of it like a small, super-fast notepad on your desk (the cache) for the information you're using right now, while your main filing cabinet (main memory) holds everything else. When you need a piece of information, you first check your notepad. If it's there, you grab it instantly. If not, you go to the filing cabinet, retrieve the relevant file (a block of data), put a copy on your notepad, and then use it. The idea is that you'll likely need that information again soon, or other parts of that same file, making your notepad a very efficient place to look.

Key aspects of cache memory:

• **Contents:** The cache holds a subset of blocks from main memory. When a word is requested that isn't in the cache, the entire block containing that word is transferred from main memory into the cache.

• **Lines and Tags:** Because the cache has fewer storage locations (called **lines**) than main memory has blocks, a line cannot be permanently assigned to a specific block. Each cache line includes a **tag** to identify which main memory block it currently holds.

• **Locality of Reference:** Cache memory works effectively due to the principle of **locality of reference**. This principle suggests that if a particular memory location is accessed, it's highly probable that nearby memory locations or that same location will be accessed again soon.

• **Multiple Levels:** Systems can have multiple levels of cache (e.g., L1, L2, L3). Each subsequent level is typically larger and slower than the previous one (L1 is the smallest and fastest, L3 is the largest and slowest).

• **Structure:** Main memory is viewed as being divided into fixed-length blocks. The cache is composed of lines, where each line holds a block of data plus a tag. Lines may also contain control bits, such as a modification bit.

• **Performance:** Cache memory significantly reduces the average time required to access data from main memory, thereby speeding up program execution.

• **Cost:** Cache memory is more expensive than main memory but less expensive than CPU registers.

3.3.1 Cache Performance

Cache performance is a crucial aspect of how efficiently the CPU can access data. When the CPU needs to read or write data, it first looks for that data in the cache.

• **Cache Hit:** If the CPU finds the requested memory location in the cache, this is called a **cache hit**. The data is then quickly read from the cache.

• **Cache Miss:** If the CPU does not find the requested memory location in the cache, this is a **cache miss**. When a miss occurs, the cache allocates a new space, copies the data from main memory into that space, and then provides the data to the CPU.

To measure cache performance, a key metric is the **Hit Ratio**. The Hit Ratio is calculated as:

```
Hit Ratio = Number of Hits / Total Number of Accesses
```

(Where Total Number of Accesses = Number of Hits + Number of Misses)

A higher Hit Ratio indicates better cache performance because it means the CPU is finding the data it needs in the fast cache more often.

We can improve cache performance by:

• Increasing the **cache block size** (the amount of data transferred at once).

• Increasing **associativity** (how flexibly data can be placed in the cache).

• Reducing the **miss rate** (the frequency of cache misses).

• Reducing the **miss penalty** (the time it takes to fetch data from main memory after a miss).

• Reducing the **time to hit** in the cache (the time it takes to find data when a hit occurs).

3.3.2 Cache Read operation

When the processor needs to read data, it sends a read address (RA) to the cache.

There are two main scenarios:

- **Cache Hit:** If the requested data is found within the cache, it's immediately delivered to the processor. In this case, the cache connects directly to the processor, and there's no activity on the system bus (the communication pathway to main memory). Think of this like finding a frequently used tool right on your workbench – no need to go to the main storage room.
- **Cache Miss:** If the requested data is not in the cache, a process is initiated to bring it in. The block of data containing the requested word is first loaded into the cache from main memory. Then, that word is delivered to the processor. The provided figure shows these actions (loading into cache and delivering to processor) happening at the same time. When a miss occurs, the address is placed on the system bus, and the data travels back through a data buffer to both the cache and the processor. In some organizations, the cache sits directly between the processor and main memory, acting as a gatekeeper for all data, address, and control signals. In such cases, a miss means the word is first brought into the cache and then passed from the cache to the processor.

Key terms:

- **Read Address (RA):** The specific location in memory that the processor wants to read data from.
- **System Bus:** A communication pathway that connects various components of the computer, including main memory.

3) Mapping Function

Since the cache has fewer lines than main memory has blocks, a method is needed to decide which main memory block can be placed in which cache line. This method is called the mapping function.

The mapping function also helps determine which main memory block is currently stored in a given cache line.

The choice of mapping function significantly influences how the cache is organized. There are three main techniques:

- **Direct Mapping:** Each main memory block can only be mapped to one specific cache line.
- **Associative Mapping:** A main memory block can be placed in any available cache line.
- **Set-Associative Mapping:** A compromise where each main memory block can be mapped to any line within a specific set of cache lines.

1) Cache Addresses

Many processors, including most non-embedded and some embedded types, utilize virtual memory.

Virtual memory is a system that allows programs to access memory using logical addresses, independent of the actual amount of physical main memory available.

When virtual memory is in use, the addresses found in machine instructions are called virtual addresses.

3.3.3 Elements of Cache Design

While there are many ways to build caches, a few core design elements help us categorize and understand different cache architectures. These elements dictate how a cache operates and interacts with the rest of the computer system.

Think of these elements like the blueprint for a small, super-fast storage room right next to your main workshop (the CPU). The blueprint tells you how big the room is, how the shelves are arranged, and how you decide what goes on which shelf. Different blueprints lead to different kinds of storage rooms, each with its own strengths.

This section will explore these key cache design parameters, sometimes referencing their use in high-performance computing (HPC) environments where speed is critical.

2) Cache Size

The ideal size for a cache is a balance. We want the cache to be large enough to significantly speed up memory access, bringing the average access time closer to that of the fast cache itself. At the same time, we want it to be small enough so that the overall average cost per bit of the combined cache and main memory system is close to the cost of main memory alone.

Minimizing cache size also has other practical benefits. A larger cache requires more complex circuitry, specifically more gates, to manage the addressing of its contents. This complexity can impact performance and cost.

5) Write Policy

When a block of data in the cache needs to be replaced with new data, the way we handle potential changes to the original data is determined by the write policy.

There are two main scenarios:

- **No changes made:** If the data in the cache block hasn't been modified since it was loaded, it can simply be overwritten with the new data without any further action. Think of it like replacing a whiteboard drawing with a new one – if the old drawing wasn't important, you just erase and redraw.
- **Changes made:** If any part of the data in that cache line has been written to (meaning it's been altered), the modified data must first be written back to main memory. Only after the main memory is updated can the old cache block be replaced with the new data. This is like carefully copying an updated document to a master file before discarding the old draft.

Different write policies exist, each with its own trade-offs in terms of performance and cost.

4) Replacement Algorithms

When a new block of data needs to be brought into the cache, and the cache is full, an existing block must be removed to make space. This decision is made by a replacement algorithm.

For direct mapping cache designs, there's no choice involved – each memory block has only one specific location in the cache. However, for associative and set-associative cache designs, a replacement algorithm is necessary.

To ensure these algorithms operate quickly, they are typically implemented in hardware.

Here are four common replacement algorithms:

- **Least Recently Used (LRU):** This is generally considered the most effective. It replaces the block that hasn't been accessed for the longest time. Think of it like clearing out the oldest items from a refrigerator first to make room for new groceries.
- **First-In, First-Out (FIFO):** This algorithm replaces the block that has been in the cache the longest, regardless of how recently it was used. It's like a queue where the first person in line is the first to be served (or, in this case, replaced).
- **Least Frequently Used (LFU):** This method replaces the block that has been accessed the fewest times. It prioritizes keeping data that is accessed often.
- **Random Replacement (RR):** As the name suggests, this algorithm randomly selects a block to replace.

7) Number of Caches

As computer processors became more powerful and logic density increased, it became possible to place a cache directly on the same chip as the processor. This is known as an **on-chip cache**.

An on-chip cache offers significant advantages over a cache that is accessed via an external bus. By being closer to the processor, it reduces the need for the processor to communicate over the external bus, which speeds up execution times and boosts overall system performance.

Initially, computer systems typically had just one cache. However, modern systems commonly employ multiple caches. This design choice involves two key considerations:

- **Number of levels of caches**: How many layers of caches are used in the memory hierarchy.
- **Unified versus split caches**: Whether the cache stores both instructions and data together (unified) or separately (split).

6) Line size

Another important design element for cache memory is the **line size**. This refers to how much data is brought into the cache when a block is retrieved. Instead of just fetching the single word the processor needs, the cache line size dictates that a group of adjacent words is also brought in.

Think of it like this: Imagine you're looking for a specific recipe in a cookbook. Instead of just tearing out the single page with that recipe, you decide to take the whole chapter it belongs to. This chapter (the block) might contain other recipes (adjacent words) that you might also want to cook soon.

• **Initial Benefit:** As the line size increases from very small, the **hit ratio** (the measure of how often the cache successfully finds the data it needs) tends to increase. This is due to the **principle of locality of reference**. This principle suggests that if a particular piece of data is accessed, it's likely that data located nearby will be accessed soon afterwards.

• **Diminishing Returns:** However, if the line size becomes too large, the hit ratio can start to decrease. This happens because the probability of actually using all the newly fetched data within that large block becomes lower than the probability of needing to reuse older data that is already in the cache and will now be replaced.